

Exact Policy-Space Credential Derivation for Legacy Password Rotation

Yuanyi Zhang

Independent Researcher

Abstract

Enterprise services often retain password-only authentication even when their users and endpoints are centrally managed. Deterministic password generators can avoid storing one password value per service, but commonly used encoders do not characterize the distribution over the complete set of policy-compliant passwords, and independent derivations may repeat during rotation. This paper presents an exact policy-space derivation algorithm that compiles a bounded password policy into a finite deterministic automaton, counts its accepted language exactly with arbitrary-precision integers, and provides a canonical rank/unrank bijection. A credential generation is then mapped through a domain-separated finite-domain permutation before unranking. The construction is an application of established regular-language ranking and rank-then-encipher format-preserving encryption; we do not claim a new FPE, KDF, or sampling primitive. Its contribution is to bind policy-space ranking to service-separated generation sequences and authenticated history exclusion. Under a total permutation abstraction, outputs comply with the policy and cannot repeat within one credential epoch before domain exhaustion; in an ideal permutation experiment, each fixed generation is uniform over the accepted language. A Rust prototype includes exact counting, rank/unrank, an FF1 cycle-walking research backend, and fixed vectors. A reproducible converter processes 270 historical website policies from the public SOUPS 2022 PCP corpus: 121 are translated exactly, 55 enter a resource-bounded compilation experiment, and all 55 compile; every exclusion has a recorded reason. A pinned probe of the official MFDPG output mechanism confirms branch bias on an enumerable 11-word language. For 500 derivations, FF1 cycle walking has median one, P95 three, and maximum six primitive calls. These results establish feasibility and expose state explosion and

backend-totality limits; they do not establish universal policy compatibility or production readiness.

Keywords: deterministic password generation, password policy, finite automata, format-preserving encryption, credential rotation

1. Introduction

Passwordless authentication is preferable when a service can be changed, but many internal Web applications, appliances, and vendor systems expose only a password interface. A client-side password manager must then retain or regenerate service credentials. Deterministic generators such as PwdHash, PALPAS, AutoPass, Spectre, and MFDPG investigate variants of this design space [1, 2, 3, 4, 5]. They reduce stored password values, but the generator must also satisfy service-specific composition rules and produce a well-defined sequence across changes in the root key, policy, account identity, and password history.

Two questions are easily blurred. First, does every random choice made by an encoder avoid modulo bias? Second, is the *final* output uniform over all strings accepted by the complete policy? The first property does not imply the second. For example, a generator that first inserts one required character per class and then shuffles can assign different masses to legal strings with different class multiplicities. A second issue appears during rotation: a deterministic function evaluated independently at generations $g_1 \neq g_2$ can return the same password. The probability may be small in a large domain, but there is no structural non-repetition guarantee.

Established work already supplies the mathematical tools needed to address these issues. Accepted words of a finite regular language can be counted and ranked, and rank-then-encipher maps arbitrary finite formats through a keyed permutation [6, 7, 8]. Our research question is therefore not whether these tools can be reinvented for passwords. It is whether they can define a deterministic, service-separated password sequence with exact policy-space uniformity and structural non-repetition, including policy changes and bounded password history.

This paper makes three scoped contributions:

1. It defines a canonical, resource-bounded policy IR and implements exact arbitrary-precision counting plus a shortlex rank/unrank bijection for the supported policy subset. Unsupported state growth fails closed.

2. It instantiates generation-indexed rank-then-encipher and binds its key and tweak to credential identity, root-key generation, policy hash, and policy epoch, yielding collision-free rotation within each epoch.
3. It integrates bounded authenticated history exclusion with the derivation sequence. In the ideal-permutation model, first-acceptable scanning is uniform over the policy language after authenticated history removal. The artifact evaluates the published MFDPG artifact, a public policy corpus, and cycle-walk availability.

The paper intentionally avoids stronger claims. The construction is not a new FPE or regular-language algorithm. The historical website corpus is not a current enterprise-policy sample. The bounded FF1 cycle-walking backend is a research instance that can fail closed after a configured number of walks.

2. Related work and novelty boundary

2.1. Deterministic password generation

PwdHash derives site-specific passwords in the browser but does not define an exact policy-space permutation [1]. PALPAS combines a high-entropy master secret with per-site salts and synchronizes non-secret parameters [2]. AutoPass studies automatic generation and changing of site passwords [3]. Spectre specifies deterministic, site-scoped password templates [4]. These systems establish that deterministic service isolation and password templates are not new contributions.

MFDPG is the closest password-policy baseline. It surveys deterministic generators, converts regular-expression policies through NFA and DFA representations, and drives an Xeger/RandExp-style generator with a deterministic HMAC-DRBG [5]. We pin its archived artifact and dependency lockfile at a named commit [9]. Its artifact selects among available regular-expression branches rather than weighting a branch by its number of accepted completions. Our distinction is not “using a DFA.” It is the exact accepted-space model and its binding to a non-repeating, generation-indexed credential sequence. This distinction is narrower than proposing a new password-generation primitive.

Gautam et al. define a machine-readable password-composition-policy language and publish the historical policy dataset used in their SOUPS 2022 study [10]. Alroomi and Li subsequently measured password-creation policies at more than 20,000 websites [11]. We use only the former authors’ public 270-policy workbook as an executable corpus; the larger measurement provides

external scope evidence but no raw policies are reconstructed from aggregate results.

2.2. Counting, ranking, and finite-domain encryption

Ranking finite-language strings and uniform generation from regular languages have a long history [6, 12]. Black and Rogaway formalized ciphers on arbitrary finite domains [7]. Bellare et al. explicitly described rank-then-encipher FPE and applied ranking and unranking to regular languages [8]. Consequently, the composition “regular language + rank/unrank + keyed permutation” is prior art. Our claims concern its password-rotation specialization: stable salt semantics, separate root and policy generations, exact non-repetition, and uniform exclusion of an authenticated history set.

NIST SP 800-38G standardizes FF1 and FF3-family format-preserving encryption; the 2025 revision draft removes FF3 and requires an FF1 domain of at least one million [13]. Small domains also require care against message-recovery and codebook-style attacks [14]. We use FF1 only as a replaceable prototype backend and enforce the draft’s minimum domain. The security statements are parameterized by a finite-domain permutation interface rather than equating the interface with this bounded implementation.

3. Model and security assumptions

The client serves legacy applications that publish a finite password policy and accept a username and password. It holds a uniformly sampled root key and authenticated, non-secret derivation metadata: service and account identifiers, a credential salt, policy identifier and hash, policy epoch, and generation. The endpoint is trusted while deriving or submitting a password; compromise of an unlocked endpoint or root key exposes derivable credentials and is outside the construction’s protection.

We assume HKDF behaves as a PRF and the selected finite-domain keyed permutation meets its stated security bounds. A compromised service learns its own password, policy, and public context but should not compute another service’s sequence. Online guessing, server-side password hashing, malicious policy changes, rollback of authenticated metadata, and availability of the remote service are outside the algorithmic claim. Password strength remains bounded by the exact accepted-space size and cannot be increased by deterministic derivation.

4. Policy model and exact bijection

4.1. Supported policy IR

A policy P defines a finite language $\mathcal{L}_P \subseteq \Sigma^*$ with bounded length $L_{min} \leq |w| \leq L_{max}$. The version-1 policy IR supports an ordered effective alphabet, forbidden characters, class minimum and maximum counts, fixed positions, fixed prefix and suffix, forbidden first and last characters, per-character maximum totals, maximum identical and sequential ASCII runs, and finite forbidden substrings. ‘null’, empty, and absent fields are distinguished by canonical JSON serialization. The alphabet order is part of the protocol.

The compiler constructs a reachable deterministic product state containing only data required by enabled constraints. Minimum counters saturate after the minimum is met; maximum counters do not. Prefix/suffix and fixed-position constraints prune transitions. A bounded suffix state recognizes forbidden substrings. The compiler rejects an empty language, inconsistent class, unsupported normalization, length above 128, or more than 250,000 reachable states. Thus “supported” means expressible by this IR *and* compilable within its explicit resource budget, not arbitrary PCRE or semantic policy code.

4.2. Exact counting

Let the compiled automaton be $A_P = (Q, \Sigma, \delta, q_0, F, L_{min}, L_{max})$. For a target length ℓ , define

$$C_\ell(i, q) = |\{x \in \Sigma^{\ell-i} : \delta^*(q, x) \in F\}|. \quad (1)$$

The boundary and recurrence are

$$C_\ell(\ell, q) = \begin{cases} 1 & q \in F \\ 0 & q \notin F, \end{cases} \quad (2)$$

$$C_\ell(i, q) = \sum_{c \in \Sigma: \delta(q, c) \neq \perp} C_\ell(i+1, \delta(q, c)). \quad (3)$$

The exact space is $N_P = \sum_{\ell=L_{min}}^{L_{max}} C_\ell(0, q_0)$. Every count is an arbitrary-precision non-negative integer. ‘entropyBits’ reports $\log_2 N_P$ as the entropy of a uniform random variable over the language; a fixed keyed derivation output is deterministic and must not itself be described as having that entropy.

4.3. Rank and unrank

Strings use shortlex order: length first, then the canonical alphabet order. To unrank $r \in [0, N_P)$, the algorithm subtracts complete length blocks, then examines each outgoing character in order. The count on that edge is the size of its contiguous rank interval. Ranking performs the inverse additions.

Lemma 1 (bijection). If the count table is correct, $\text{Unrank}_P : [0, N_P) \leftrightarrow \mathcal{L}_P$ is a bijection and Rank_P is its inverse.

Proof sketch. At each state, the recurrence partitions the completion set into disjoint, adjacent intervals indexed by the next character. Their sizes sum to the current completion count. Induction on the remaining length gives existence and uniqueness; the outer shortlex length blocks are disjoint by construction. $\square$

5. Generation-indexed finite-domain derivation

The Root Key first yields a vault-scoped subkey. A 256-bit credential key is then derived with HKDF using the 128-bit credential salt and canonical context containing ‘serviceID’ and ‘accountID’:

$$K_{\text{vault}} = \text{HKDF}(K_{\text{root}}, \text{policy-space-credential-derivation}), \quad (4)$$

$$K_{\text{cred}} = \text{HKDF}(K_{\text{vault}}, \text{credentialSalt}, \text{Canonical}(\text{serviceID}, \text{accountID})). \quad (5)$$

The exact byte-level construction is specified by canonical JSON contexts and the artifact’s fixed vector. The permutation tweak binds the vault, service, account, credential salt, Root-Key generation, policy identity, policy hash, policy epoch, derivation version, and encoder version. Credential generation is deliberately excluded from the tweak because it is the permutation input.

Let $\Pi_{K,T}$ be a total keyed permutation on $[0, N_P)$. For generation $0 \leq g < N_P$,

$$\text{Derive}(K, T, g, P) = \text{Unrank}_P(\Pi_{K,T}(g)). \quad (6)$$

The prototype implements the interface using FF1-AES-256 over the smallest binary superset followed by cycle walking. It rejects domains below one million, domains above 512 bits, inputs outside the domain, and derivations that exceed 1,024 walks. This limit makes the backend a partial, failure-closed

implementation. The following properties therefore refer to the abstract total permutation, or to successful calls of the prototype where stated.

Let $M = 2^{\lceil \log_2 N_P \rceil}$. Since $N_P > M/2$ for a non-power-of-two domain, a permutation orbit can visit at most $M - N_P$ outside points before returning to the accepted subdomain. Thus unbounded cycle walking terminates within $M - N_P + 1$ primitive calls. For an ideal random permutation,

$$\Pr[W > k] = \frac{(M - N_P)_k}{(M)_k} \leq \left(\frac{M - N_P}{M} \right)^k < 2^{-k}, \quad (7)$$

where $(x)_k$ is a falling factorial. The 1,024-call cap therefore has an ideal-model tail below 2^{-1024} ; it remains a partial concrete interface.

Theorem 1 (compliance). Every successful derivation belongs to $\mathcal{L}_P$. This follows directly from Lemma 1.

Theorem 2 (epoch-local non-repetition). For fixed (K, T, P) and $g_1 \neq g_2$ in the domain, successful outputs differ. Both Π and Unrank are injective.

Theorem 3 (ideal-permutation marginal). If Π is sampled uniformly from all permutations on $[0, N_P)$, then for fixed g and any $w \in \mathcal{L}_P$, $\Pr[\text{Derive}(K, T, g, P) = w] = 1/N_P$. A real PRP gives computational indistinguishability under its security assumptions, not unconditional exact uniformity over keys.

Theorem 4 (context separation). Under HKDF and tweakable-PRP assumptions, distinct credential identities, salts, Root-Key generations, or policy epochs induce computationally separated derivation contexts. This is a computational claim; outputs from different generations in one permutation are sampling without replacement and are not statistically independent.

6. History exclusion across policy epochs

A same-policy rotation increments the epoch-local generation under one permutation. A policy change increments the epoch and resets generation to zero; the policy hash selects a different permutation. The credential salt is stable within this lineage. Generating a fresh salt at every rotation would select unrelated permutations and destroy Theorem 2’s sequence interpretation.

Independent policy epochs can have intersecting languages, so Theorem 2 does not prevent cross-epoch reuse. For a server history window h , the client retains authenticated descriptors of the recent committed versions and

rederives their passwords; it does not store the values. Let E be the set of rederived recent passwords. The pending epoch tests successive unused generation inputs and selects the first output outside E .

Proposition 1 (bounded exclusion). Under a total permutation, if at least $|E \cap \mathcal{L}_P| + 1$ unused generation inputs remain, the process terminates after at most $|E \cap \mathcal{L}_P| + 1$ candidates. It fails when history cannot be authenticated or rederived, including history under an unavailable Root-Key generation. A configured history window above 64 is rejected.

Theorem 5 (uniform exclusion marginal). Let $e = |E \cap \mathcal{L}_P| < N_P$. Under an ideal random permutation, scanning distinct generation inputs and returning the first output outside E produces every $w \in \mathcal{L}_P \setminus E$ with probability $1/(N_P - e)$. It uses at most $e + 1$ attempts and has expected attempt count $(N_P + 1)/(N_P - e + 1)$.

Proof sketch. The scanned images form a uniformly ordered sample without replacement. Symmetry makes every non-excluded rank equally likely to be the first non-excluded element; at most all e excluded ranks precede it. The expectation is the first-success order statistic. This statement concerns the ideal model and authenticated local history, not undocumented server state.

The client-side check cannot predict undocumented server history rules. Such a server rejection is outside the policy-space algorithm and must not silently change the committed generation metadata.

7. Implementation

The Rust prototype separates policy compilation from finite-domain derivation. The policy compiler creates only reachable product states and stores all completion counts as ‘BigUint’. Public operations include exact count by length, total count, rank, unrank, and policy hash. Property tests compare enumeration and counting, exercise ‘Rank(Unrank(r))=r’, test values above 128 bits, and verify constraint families.

‘DomainPermutation’ isolates the mathematical construction from the FF1 research backend. The fixed vector binds credential context, salt, policy descriptor, root key, credential key, exact domain size, rank, and output password. Callers may cache a compiled policy; complete derivation otherwise compiles on demand. The history selector rederives the authenticated exclusion window and consumes successive generation inputs until it finds the first non-excluded output.

Table 1: SOUPS 2022 PCP corpus translation and bounded compilation.

Outcome	Policies
Public historical source records	270
Exactly translated to the current IR	121
Compiled with evaluation $L_{max} \leq 32$	55
Translated but resource-skipped ($L_{max} > 32$)	66
Rejected for unsupported semantics	149
Compilation rejection among evaluated policies	0

The current implementation limitation is that an arbitrary finite-domain PRP backend with a proof-matched, total implementation has not replaced bounded FF1 cycle walking. Complex forbidden-substring compilation is also a simple suffix-state construction rather than a minimized Aho–Corasick product, which explains one stress-policy rejection.

8. Evaluation

All measurements are reproducible release-mode local baselines. They were run on one macOS x86-64 host. Raw JSON contains timestamps, sample counts, exact integers, and claim boundaries.

8.1. Policy-corpus coverage and exact spaces

The public SOUPS 2022 workbook contains 270 historical website policies. Our converter records the workbook SHA-256, source row, original PCP JSON, translation result, and reason for every rejection. It exactly translates 121 policies to the current IR. To bound dynamic-programming memory in this run, only the 55 translated policies with $L_{max} \leq 32$ are compiled; all 55 compile. The median state count is 31 and the maximum is 23,227. Median compile time is 9.08 ms, P95 is 136.84 ms, and one 12.2-million-cell case takes 13.7 s. The other 66 translated policies are resource-skipped, not classified as unsupported. Table 1 gives the complete accounting.

The separate synthetic corpus contains 12 representative stress policies. Eleven compile within 250,000 states. Common class constraints require 9–254 states; edge and run constraints require 6,201–8,889 states; per-character totals reach 159,250 states. A stress case with long forbidden substrings over a 36-character alphabet is rejected at the state limit. Table 2 reports selected cases.

Table 2: Selected synthetic policy compilation results.

Policy	States	Compile (μs)	$\log_2 N_P$
Lower/digit, length 8	9	999	41.36
Four classes, length 12	164	3,644	71.36
Four classes, length 18	254	6,972	111.60
No symbol at edges, length 16	7,531	170,006	95.63
Per-character total ≤ 2	159,250	721,724	34.91
Long forbidden-substring stress	rejected	386,308	—

Table 3: Distribution over the same 11-word regular language (100,000 samples).

Method	Empirical TVD	χ^2	max/min count
Proposed count-rank-unrank	0.00479	15.43	1.04
Published MFDPG artifact [5]	0.40730	200,737	10.15

For the 18-character, 74-symbol case, the exact count is

$$N_P = 3923998958022941748259805567385600,$$

corresponding to 111.596 bits for a uniform variable over that language.

8.2. Distribution and collision experiments

The published MFDPG artifact provides the only comparative implementation in this experiment. We pin its repository commit and locked dependencies, then evaluate its official RandExp selection path and our algorithm on the same enumerable regular language $\{a, b0, \dots, b9\}$, for which $N_P = 11$. Each method receives 100,000 labelled deterministic seeds. Table 3 reports empirical distance to the uniform target.

MFDPG produces ‘a’ 49,821 times and each ‘b’ string roughly 5,000 times, matching the analytic equal-branch TVD $9/22 \approx 0.40909$. The proposed algorithm’s TVD is 0.00479 and its $\chi^2 = 15.43$ for ten degrees of freedom. The MFDPG probe replaces only its Argon2id preimage step while retaining the published artifact’s exact output-selection path; it is not a latency comparison. The proposed FF1 backend also yields zero observed collisions over 2,000 generations in a large policy domain. The non-repetition claim follows from injectivity, not from this sample.

8.3. Local performance

For the 18-character policy, 500 operations give rank and unrank medians near 1–2 microseconds. Complete proposed derivation, including HKDF, canonical tweak, FF1 cycle walking, and unrank, has median 80.4 microseconds, P95 205.0 microseconds, and P99 329.5 microseconds. Cycle walking has mean 1.348 calls, median one, P95 three, P99 four, and maximum six. These values exclude storage, network, and user delay.

9. Security discussion

The principal unconditional properties are language compliance and non-repetition conditioned on successful execution of a permutation. Exact space size is a combinatorial fact. Uniformity and context separation require ideal-permutation or computational assumptions, respectively. Password outputs remain limited by N_P and by server-side online/offline attack conditions; deterministic generation cannot add entropy to a small policy language.

Root-Key compromise reveals every derivable credential in its generation. Credential salt is not a secret and prevents neither Root-Key compromise nor malicious endpoint derivation. Its role is domain separation and stable lineage identity. Changing it intentionally creates a new lineage and ends the within-permutation non-repeat scope.

10. Limitations and future work

The most important limitations are empirical and cryptographic. The policy corpus is historical Web data rather than a current sample of enterprise deployments. The official MFDPG comparison isolates an enumerable distribution counterexample rather than testing every shared real-world policy. The compiler can suffer product-state explosion and currently lacks automaton minimization and optimized multi-pattern matching. Unicode normalization, dictionary or breached-password services, context-dependent remote rules, and unbounded semantic predicates are not supported.

The FF1 backend follows a draft minimum-domain rule and bounds cycle walking. A production instantiation should use a reviewed, proof-matched permutation whose implemented domain and failure semantics exactly match the security statement, or prove availability bounds acceptable to the application.

The evaluation lacks Windows and Linux measurements, a broader current policy corpus, comparison on a larger shared policy subset, and an independent

cryptographic review of the concrete backend. These are necessary before a production-readiness claim.

A separate theoretical question remains open for this project: dynamic, intersecting policy languages under adversarial policy changes and a bounded history state. Before treating that as a new mathematical contribution, it requires an additional prior-art audit across dynamic FPE, constrained sampling without replacement, and state lower bounds.

11. Conclusion

Exact policy-space derivation can replace an ad hoc composition encoder without inventing new cryptographic primitives. The construction compiles a bounded policy, counts and bijectively indexes every accepted password, and maps credential generations through a domain-separated finite-domain permutation. Its contribution is a credential-specific construction around known rank-then-encipher tools: exact policy compilation, service-separated generation permutations, collision-free epoch rotation, and uniform history exclusion in the ideal model. The prototype and experiments show feasibility and make state explosion, backend totality, and deployment gaps measurable. Those boundaries are central to the claim: the work is an evidence-backed research prototype for legacy password systems, not a new FPE or a finished production password platform.

Artifact availability

The artifact contains the Rust implementation, fixed derivation vector, public-corpus converter and provenance, pinned MFDPG probe, raw JSON measurements, and reproduction commands. An anonymized repository URL will be added for review.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence this work.

Funding

This research received no specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data availability

The evaluation uses the public SOUPS 2022 PCP dataset and records its source URL and workbook digest. Derived corpus rows, raw measurements, models, and reproduction scripts are included in the review artifact; an anonymized URL will be inserted before submission.

CRedit authorship contribution statement

Yuanyi Zhang: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing—original draft, Writing—review and editing, Visualization.

Declaration of generative AI and AI-assisted technologies in the writing process

During preparation of this work, the author used ChatGPT and Codex to support language editing, code inspection, and test and documentation drafting. The author reviewed and edited all resulting material and takes full responsibility for the content of the publication.

References

- [1] B. Ross, C. Jackson, N. Miyake, D. Boneh, J. C. Mitchell, Stronger password authentication using browser extensions, in: 14th USENIX Security Symposium, 2005.
- [2] M. Horsch, A. Hulsing, J. Buchmann, Palpas—passwordless password synchronization, in: 10th International Conference on Availability, Reliability and Security, 2015, pp. 30–39. doi:10.1109/ARES.2015.23.
- [3] F. A. Maqbali, C. J. Mitchell, Autopass: An automatic password generator, arXiv:1703.01959 (2017).
URL <https://arxiv.org/abs/1703.01959>
- [4] M. Billefont, Spectre: A password cipher (2023).
URL <https://spectre.app/>
- [5] V. Nair, D. Song, MFDPG: Multi-factor authenticated password management with zero stored secrets, in: Network and Distributed System Security Symposium, 2024. doi:10.14722/ndss.2024.24646.

- [6] A. V. Goldberg, M. Sipser, Compression and ranking, in: 17th Annual ACM Symposium on Theory of Computing, 1985, pp. 440–448.
- [7] J. Black, P. Rogaway, Ciphers with arbitrary finite domains, in: Topics in Cryptology—CT-RSA 2002, 2002, pp. 114–130.
- [8] M. Bellare, T. Ristenpart, P. Rogaway, T. Stegers, Format-preserving encryption, in: Selected Areas in Cryptography, 2009, pp. 295–312.
URL <https://eprint.iacr.org/2009/251>
- [9] V. Nair, D. Song, MFDPG reference artifact, archived repository, commit 6c26096dd22ff2b18aa5d8e4c3d5b0caf7b45bb7 (2023).
URL <https://github.com/multifactor/mfdpg>
- [10] A. Gautam, S. Lalani, S. Ruoti, Improving password generation through the design of a password composition policy description language, in: Eighteenth Symposium on Usable Privacy and Security, 2022.
URL <https://www.usenix.org/conference/soups2022/presentation/gautam>
- [11] S. Alroomi, F. Li, Measuring website password creation policies at scale, in: ACM SIGSAC Conference on Computer and Communications Security, 2023. doi:10.1145/3576915.3623156.
- [12] J.-B. Oudinet, A. Denise, A. Genitrini, A new dichotomic algorithm for the uniform random generation of words in regular languages, Theoretical Computer Science 502 (2013) 165–176. doi:10.1016/j.tcs.2012.07.025.
- [13] National Institute of Standards and Technology, Methods for format-preserving encryption: SP 800-38G revision 1, second public draft, Tech. rep., NIST (2025).
URL <https://csrc.nist.gov/pubs/sp/800/38/g/r1/2pd>
- [14] F. B. Durak, S. Vaudenay, Breaking the FF3 format-preserving encryption standard over small domains, in: Advances in Cryptology—CRYPTO 2017, 2017.